

Security, Privacy and Threat Model

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-12
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define protected assets, trust boundaries, threats, controls, privacy behavior, secure development and incident response.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Security and privacy posture

Media Reminder is intentionally offline and account-free. This reduces remote attack surface but does not eliminate risk: the app processes untrusted media and ZIP archives, exposes notification actions, holds private personal content, and interacts with Android components while the process may be absent.

The production build MUST have no Internet permission. Android automatic cloud backup and device-transfer extraction of app data are explicitly excluded so imported media is not silently moved outside the user's manual export workflow.

2 Protected assets

Asset	Sensitivity	Protection objective
Imported media	Potentially highly private	Confidential inside Android sandbox; never upload; controlled sharing only
Reminder labels, notes and schedules	Private behavioral data	Least lock-screen disclosure; local storage; backup warning
Backup archives	Plaintext copy of private data	Explicit export, validation, scoped URI grants, clear custody warning
Alarm actions	Integrity-critical	Authenticated session/nonce, immutable PendingIntents, idempotent transitions

Asset	Sensitivity	Protection objective
Scheduler state	Integrity/availability-critical	Room + generation validation; no arbitrary external control
Signing key	Release-critical secret	Offline/secure storage outside repository and CI logs
Source and dependencies	Supply-chain asset	Review, pinning, SBOM, license and vulnerability process
Diagnostics	Potential metadata	Bounded, redacted and user-previewed before sharing

3 Trust boundaries

1. **User and Android system UI** - system picker/settings/notification surfaces are trusted to grant scoped choices.
2. **External content provider** - selected media/document URI is untrusted input even when chosen by the user.
3. **App sandbox** - Room and app-owned files are trusted only after integrity validation.
4. **Backup archive** - completely untrusted until staged and validated.
5. **Android IPC** - broadcasts, intents, PendingIntents and FileProvider URIs require explicit package/token checks.
6. **React Native bridge** - internal but treated as a typed boundary; JavaScript cannot request arbitrary file/system operations.
7. **Release pipeline** - build workers and dependency registries are supply-chain boundaries.

4 Threat actors and misuse

- another installed app sending crafted explicit/implicit intents;
- a malicious or malformed media file exploiting parser behavior;
- a malicious backup intended for traversal, resource exhaustion or record injection;
- accidental sharing of a plaintext archive;
- a person with unlocked physical device access;
- compromised dependency or build pipeline;
- developer mistakes such as exported receiver, mutable PendingIntent, sensitive log or broad URI grant;
- OEM/platform behavior that exposes more notification text than expected.

The product does not attempt to defend against a fully compromised operating system or an attacker who controls an unlocked device and can use the app normally. It does reduce accidental disclosure and cross-app manipulation.

5 Privacy requirements

- **SEC-001:** No production code path may send network traffic; manifest lacks `INTERNET` and `ACCESS_NETWORK_STATE` unless a future ADR changes scope.
- **SEC-002:** No account, advertising identifier, analytics SDK, crash-upload SDK or remote configuration.
- **SEC-003:** Media, labels, notes and schedule values **MUST NOT** appear in diagnostic export.
- **SEC-004:** Lock-screen notification visibility defaults to Private; user may choose generic Secret content.
- **SEC-005:** Automatic Android cloud backup and device transfer **MUST** exclude Room, DataStore, app media, diagnostics and device-protected schedule envelope.
- **SEC-006:** User-initiated ZIP export **MUST** warn that v1 archives are readable by anyone who obtains them.
- **SEC-007:** Sharing uses the Android chooser and read-only, temporary URI grants; no public filesystem directory is exposed.
- **SEC-008:** Clearing history must not delete reminder definitions, and deleting a reminder must not imply secure erasure from flash backups.

6 Android backup exclusion

Use both legacy and modern controls appropriate to supported APIs:

- `android:allowBackup="false"` as a conservative baseline;
- `android:dataExtractionRules` excluding `root`, `file`, `database`, `sharedpref`, `external` and device-protected domains from cloud backup and device transfer;
- legacy `fullBackupContent` exclusion file for older APIs;
- automated APK/manifest test and restore test to confirm no app-owned media appears through platform backup paths.

The manual `.mrbackup.zip` format is the supported migration mechanism. If a future release opts into operating-system transfer, it requires a privacy ADR and user-facing disclosure.

7 Exported component policy

All receivers, services, activities and providers are `android:exported="false"` unless platform integration strictly requires otherwise. `FileProvider` is exported false and grants URI permissions. Intent filters are minimized. Runtime handlers verify:

- action string is exact;
- package/component is expected;
- session/operation ID exists;
- nonce/token is valid and unexpired;
- current database state permits the transition;
- extras have type/length bounds.

Any impossible intent stops active sound where relevant, logs a safe diagnostic and returns without mutation.

8 PendingIntent security

PendingIntents are immutable by default. Use explicit package/component. Request codes are generated through a collision-tested factory, and semantic authorization still comes from session/nonce in Room, not obscurity of request code. FLAG_UPDATE_CURRENT is used only with immutable payload ownership; stale generations are rejected.

9 File and URI security

- System picker URIs are read only.
- Persistable grants are used only for custom user-selected tone where necessary; imported media is copied and the grant released.
- User display names are metadata, not paths.
- Internal filenames are UUIDs with a normalized allowlisted extension.
- Path resolver canonicalizes and confirms target remains inside expected root.
- FileProvider exposes only completed export paths declared in XML, never database/media roots broadly.
- Temporary grants are revoked or naturally expire after share; no write grants.

10 Media input handling

Media is untrusted. Controls:

- stream copy with size limits;
- MIME header sniff plus platform probe, not extension alone;
- image bounds decode before full decode;
- no embedded HTML/SVG/script execution in v1;
- Media3/platform codecs kept current through dependency updates;
- preview and thumbnail generation catch decoder failures and enforce pixel/duration limits;
- malformed content cannot change reminder database state until source copy and validation complete;
- fuzz/crash fixtures are retained in security tests when licenses permit.

A codec exploit can still exist in the platform/library. Dependency and Android security update guidance is included in release notes.

11 Backup attack controls

The archive reader treats every byte as hostile:

- no direct extraction before central-directory inspection;
- reject absolute paths, traversal, NUL, symlink, duplicate and case-collision entries;
- canonical destination check after normalization;
- entry count, size, ratio, JSON depth and string limits;
- stream hashing with bounded buffers;
- schema allowlists and enum validation;

- reference graph validation;
- staged token binds to archive digest;
- commit through repositories, never SQL supplied by archive;
- no recursive nested archive extraction;
- current data unchanged until complete validation and reviewed conflict plan.

12 Database integrity and injection

Room parameterized queries are mandatory. Search uses bound parameters and escaped wildcard semantics. There is no dynamic SQL from archive or user strings. Check constraints and domain validation defend against impossible values. Database files are never shared.

13 Lock-screen and screen capture

Default notification content is private. The user can set **Hide names on lock screen**, producing generic title and no thumbnail. The direct-boot envelope contains only opaque IDs and next due instant.

A privacy setting MAY enable FLAG_SECURE for media preview/player and blur sensitive content in Recents. It is not forced globally because users may intentionally capture their own content and portfolio screenshots. Alarm controls never expose notes on the lock screen.

14 Backup confidentiality

V1 ZIP is intentionally plaintext for reliability and portability. The export confirmation uses direct language and requires a user-selected destination. The app does not remember or auto-export to a cloud folder. Encryption is P1 and must use a reviewed format with authenticated encryption, memory-hard password derivation and no recoverable developer key.

15 Local encryption

Android file-based encryption and app sandbox are the v1 at-rest baseline. Database-level encryption is not required for MVP because it adds key management, migration and native risk while unlocked app content remains accessible. A future optional app lock/encrypted vault requires a separate ADR and must preserve alarm operation without exposing media before unlock.

16 Logging and diagnostics

Release logs use structured codes. Forbidden fields: asset title, file name/path, notes, category/tag names, ZIP destination, exact schedule label and raw URI. Allowed: UUID hashed/truncated for correlation, phase, duration, byte count, OS/API, device model, capability enum, error code and app version.

Logcat output is minimal in release. Diagnostic export is previewed, bounded and initiated by the user. Crash uploads are absent.

17 Supply-chain security

- lock JavaScript and Gradle dependency versions;
- verify checksums/signatures through supported package mechanisms;

- prefer primary AndroidX and React Native dependencies;
- generate an SBOM for each release;
- run dependency vulnerability and license scans in CI;
- remove unused transitive libraries and permissions;
- protect branches/tags and require review for workflow/signing changes;
- use reproducible build inputs where possible;
- release signing key never enters source control, chat prompts, screenshots or public CI logs.

18 Threat matrix

Threat	Control	Residual risk
ZIP traversal	canonical path validation, no direct extraction, malicious fixtures	parser/platform implementation flaw
ZIP bomb	entry/total/ratio limits, streaming, preflight space	crafted archive near limits consumes time
Intent replay	immutable explicit PendingIntent, nonce, Room idempotency	device compromise
Unlocked full-screen annoyance	trigger-time state check, no overlay, FSI omitted when unlocked	OEM notification behavior varies
Private lock-screen disclosure	private/secret visibility, generic direct boot	user/system settings can expose notification
Backup accidentally shared	clear warning, chooser, no auto-upload	user can still select public destination
Media parser exploit	allowlist, size/probe, updated Media3/platform	zero-day codec flaw
Data loss during restore	staging, checksums, rollback journal	device/storage failure during both data and rollback
Automatic cloud upload	backup/data extraction exclusions	OEM nonconformance must be tested
Dependency compromise	pinning, SBOM, review, scans	registry/build environment compromise

19 Privacy notice outline

The shipped privacy notice states:

- what is stored: imported content, reminder settings, optional history and local diagnostics;

- where: app storage on the device and user-selected backup destinations;
- what is transmitted: nothing by the app;
- permissions and their purpose;
- Android/OEM ownership of notification and alarm settings;
- deletion behavior, uninstall and backup custody;
- open-source repository and contact route without requiring personal data.

20 Security testing and response

Critical controls are covered by static manifest inspection, exported-component tests, PendingIntent replay tests, archive fuzz fixtures, path traversal tests, dependency scans and manual privacy review. A private vulnerability reporting method is documented in `SECURITY.md`. Security releases revoke compromised artifacts through release notes, publish new checksums, and never request users send private backup/media unless they have independently redacted it.

21 Security release gate

Release is blocked by any unreviewed exported component, Internet/overlay/broad storage permission, plaintext sensitive log, bypassable archive validation, missing rollback on Replace, mutable externally reachable PendingIntent, or inability to stop ringing after invariant failure.

21.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.